

```
function Product(id, name) {
  this.id = id;
  this.name = name;
  // methods which will be allocated for every object
  this.getName = function() {
    return this.name;
  }

  this.setName = function(n) {
    this.name = n;
  }
}
```

Every object has a copy of it

```
var p1 = new Product(13, 'Sony Bravia');
```

HEAP

|    |      |         |         |
|----|------|---------|---------|
| id | name | getName | setName |
|----|------|---------|---------|

```
var p2 = new Product(62, 'LG Joy');
```

|    |        |         |         |
|----|--------|---------|---------|
| 62 | LG Joy | getName | setName |
|----|--------|---------|---------|

```
var product = {
  "id": 52,
  "title": "Wacom",
  "getTitle": function() {
    return this.title;
  }
}
console.log(product.title); // Wacom
console.log(product.getTitle()); // Wacom

var ref = product.getTitle;

console.log(ref());
```

this  
↓  
window

Context is LOST

HEAP

|    |       |          |
|----|-------|----------|
| 52 | Wacom | getTitle |
|----|-------|----------|

```
var numbers = [5,2,11,87,23,60,42,12];
var evens = filter(numbers, function(no) { return no % 2 === 0});

function filter(elems, predicateFn) {
  var result = [];
  for(var i = 0; i < elems.length; i++) {
    if(predicateFn(elems[i])) {
      result.push(elems[i]);
    }
  }
  return result;
}
```

```
function greet(msg) {
  return function(name) {
    return msg + name;
  }
}
```

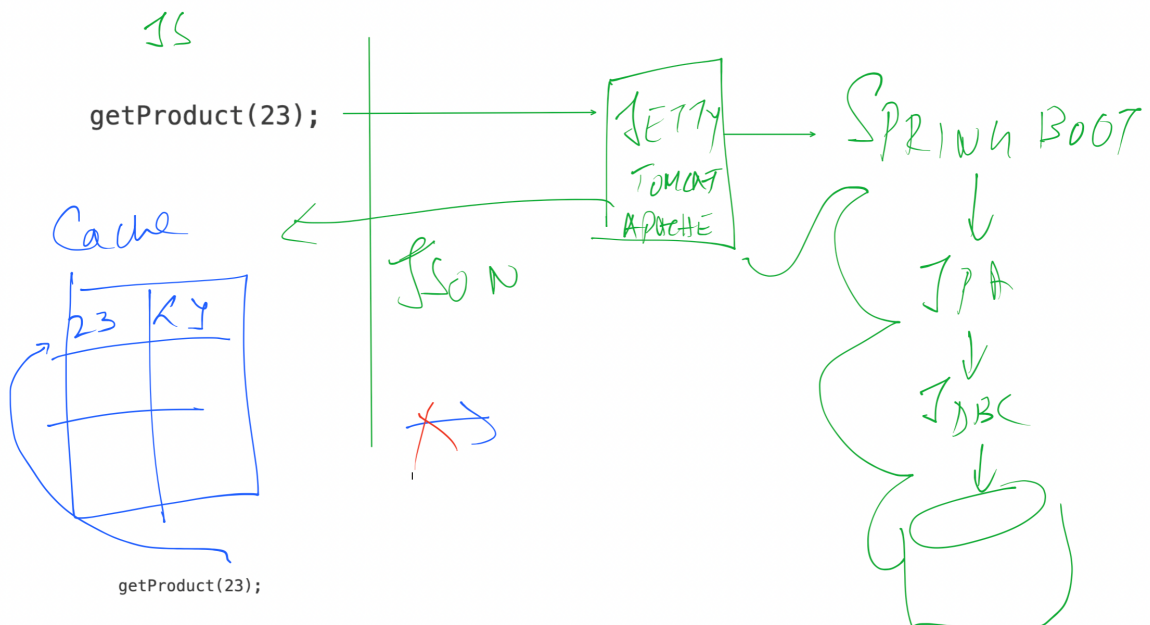
Good morning  
msg ②

} closure

```
var morningGreet = greet("Good Morning");
```

```
console.log(morningGreet("Ria"));
console.log(morningGreet("Priya"));
```

```
morningGreet.prototype
  {}
  constructor: f (name)
    length: 1
    name: ""
  prototype: {}
    arguments: null
    caller: null
    [[FunctionLocation]]: VM1667:2
  [[Prototype]]: f ()
  [[Scopes]]: Scopes(3)
    0: Closure (greet) ① ②
      msg: "Good Morning"
    [[Prototype]]: Object
    1: Script {mobiles: Array(2), data: Array(8)}
    2: Global {window: Window, self: Window, document: document, name: '', location: Location, ...}
```



```
> memFib.prototype
```

```
<  { } ⓘ  
  ▼ constructor: f (arg)  
    length: 1  
    name: ""  
  ▶ prototype: {}  
    arguments: null  
    caller: null  
    [[FunctionLocation]]: closure.html:13  
  ▶ [[Prototype]]: f ()  
  ▼ [[Scopes]]: Scopes[2]  
    ▼ 0: Closure (memo)  
      → ▶ cache: {40: 102334155}  
        ▶ fn: f fibanocci(no)  
        ▶ [[Prototype]]: Object
```

```
function fibanocci(no) {  
  if(no === 0 || no === 1) {  
    return no;  
  } else {  
    return fibanocci(no - 1) + fibanocci(no - 2);  
  }  
}
```

```
function memo(fn) {  
  var cache = {};  
  return function(arg) {  
    if(!cache[arg]) {  
      cache[arg] = fn(arg);  
    }  
    return cache[arg];  
  }  
}
```

40

fibanocci

```
var memFib = memo(fibanocci);  
  
console.time("first");  
  console.log(memFib(40));  
console.timeEnd("first");
```